TASK 12: Re-evaluate and Adapt the Design

Name: Nagineni Sunil Kumar
Register Number: 192472330

1. Aim

To evaluate and extend the original metro fare system to support the Airport Zone, a weekend discount, and monthly passes.

2. Changed Requirements

- Airport Zone fare: ₹60.
- Weekend discount: 10%.
- The weekend discount must not be combined with the peak surcharge.
- A monthly pass allows unlimited travel in two selected zones.
- The minimum-balance rule continues to apply to journeys paid for using stored-value balance.

3. Impact Analysis

Component	Required Change
Zone enumeration	Add Airport
Fare table	Add ₹60 for Airport
Fare calculation	Add weekend discount
Peak-hour logic	Keep existing boundaries
SmartCard class	Continue to protect balance
Pass class	Add two selected zones and validity
Test cases	Add Airport, weekend, pass, and balance cases

4. Evaluation of the Original Design

A switch-based fare calculation can support the new Airport Zone by adding another case. However, repeated switch statements may need to be updated in several places.

A central fare table is easier to maintain because each zone's base fare is stored in one place. The SmartCard class can still manage the balance, while a separate pass class handles travel covered by a monthly pass.

5. Revised C++ Program

```
#include <iostream>
#include <iomanip>
using namespace std;
enum class Zone {
   Inner,
   Middle,
   Outer,
   Airport };
constexpr long long FARES[] = {
   1000, 2000, 3000, 6000
};
constexpr long long MIN_BALANCE = 5000;
bool validZone(Zone z) {
   switch (z) {
      case Zone::Inner:
      case Zone::Middle:
      case Zone::Outer:
      case Zone::Airport:
```

```
        return true;  }
    return false; }
long long baseFare(Zone z) {
    if (!validZone(z))
        return -1;
    return FARES[static_cast<int>(z)];}
bool isPeak(int hour) {
    return (hour >= 8 && hour < 10) ||
           (hour >= 17 && hour < 20);}
class MonthlyPass {
private:
    Zone first;
    Zone second;
    bool valid;
public:
    MonthlyPass(Zone a, Zone b, bool active)
        : first(a), second(b), valid(active) {}
    bool covers(Zone z) const {
        return valid && (z == first || z == second);    }};

class SmartCard {
private:
    long long balance;
public:
    explicit SmartCard(long long initialBalance)
        : balance(initialBalance >= 0 ? initialBalance : 0) {}
```

```
long long getBalance() const {
    return balance;   }
bool travel(Zone zone, int hour, bool weekend,
            const MonthlyPass* pass = nullptr) {
    if (hour < 0 || hour > 23) {
        cout << "Invalid hour\n";
        return false;     }
    long long fare = baseFare(zone);
    if (fare < 0) {
        cout << "Invalid zone\n";
        return false;   }
    if (pass != nullptr && pass->covers(zone)) {
        cout << "Monthly pass accepted. Fare Rs 0.00\n";
        cout << "Balance Rs " << balance / 100 << '.'
             << setw(2) << setfill('0') << balance % 100
             << setfill(' ') << '\n';
        return true; }
    if (weekend) {
        fare = fare * 90 / 100;
    } else if (isPeak(hour)) {
        fare = fare * 120 / 100;     }
    if (balance - fare < MIN_BALANCE) {
        cout << "Denied (low balance)\n";
        return false;   }
    balance -= fare;
    cout << fixed << setprecision(2);
```

```
        cout << "Fare Rs " << fare / 100 << '.'
             << setw(2) << setfill('0') << fare % 100
             << setfill(' ') << '\n';
        cout << "Balance Rs " << balance / 100 << '.'
             << setw(2) << setfill('0') << balance % 100
             << setfill(' ') << '\n';
        return true;    }};
int main() {
    SmartCard card(15000);
    cout << "Airport weekday peak journey:\n";
    card.travel(Zone::Airport, 18, false);

    cout << "\nInner Zone weekend journey:\n";
    card.travel(Zone::Inner, 12, true);
    cout << "\nMonthly pass journey:\n";
    MonthlyPass pass(Zone::Inner, Zone::Middle, true);
    card.travel(Zone::Middle, 9, false, &pass);
    return 0;}
```

6. Expected Output

Airport weekday peak journey:

Fare Rs 72.00

Balance Rs 78.00

Inner Zone weekend journey:

Fare Rs 9.00

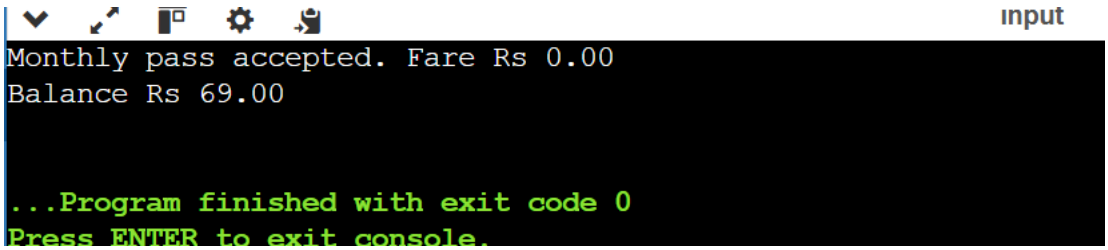
Balance Rs 69.00

Monthly pass journey:

Monthly pass accepted. Fare Rs 0.00

Balance Rs 69.00

Output:



7. Worked Examples

Example 1: Airport, weekday peak

Base fare = ₹60.

Peak surcharge = 20%.

Final fare = ₹60 × 1.20 = ₹72.

Example 2: Inner Zone, weekend

Base fare = ₹10.

Weekend discount = 10%.

Final fare = ₹10 × 0.90 = ₹9.

The peak surcharge is not added because the weekend discount has priority.

Example 3: Monthly pass

If a valid pass covers the Middle Zone, the journey fare is ₹0. The stored-value balance remains unchanged.

8. Test Cases

Test	Condition	Expected Result
1	Airport, weekday, 18:00	₹72 fare
2	Inner, weekend	₹9 fare
3	Valid pass covers selected zone	₹0 fare
4	Pass does not cover zone	Apply normal fare rules
5	Invalid hour	Reject journey
6	Fare would leave less than ₹50	Deny journey

9. Complexity Analysis

Each fare calculation, zone lookup, and pass check takes O(1) time. The program uses O(1) extra space per journey. Processing (n) journeys takes O(n) time.

10. Evidence-Based Defence

The revised design is preferable because the fare table stores all four zone fares in one place, while the monthly pass is handled separately. The weekend check comes before the peak check, ensuring that the discount and surcharge are not combined.

The expected examples demonstrate the intended rules. Compile and run the program, then include your actual output screenshot and any additional boundary tests as evidence.

11. Conclusion

The design can be modified to support the Airport Zone, weekend discounts, and monthly passes. Keeping fare calculation, card balance management, and pass validation separate makes future changes easier to implement and test.